Program 1

Install Hadoop and Implement the following file management tasks in Hadoop:
Adding files and directories
Retrieving files
Deleting files and directories.

Steps for installing Hadoop on Ubuntu

Step 1: Update Ubuntu Packages

Updates package lists and installs latest security updates.

Commands:

```
sudo apt update
sudo apt upgrade -y
```

Step 2: Install Java, SSH and wget

Hadoop requires Java, SSH for daemon communication, wget to download Hadoop.

Commands:

```
sudo apt install openjdk-11-jdk openssh-server openssh-client wget -y
java -version
```

Step 3: Configure Passwordless SSH

Enables Hadoop daemons to communicate without passwords.

Commands:

```
ssh-keygen -t rsa -P ""
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost
exit
```

Step 4: Download Hadoop

Downloads and installs Hadoop 3.3.6.

Commands:

cd /tmp
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
sudo tar -xzf hadoop-3.3.6.tar.gz -C /usr/local/
sudo mv /usr/local/hadoop-3.3.6 /usr/local/hadoop
sudo chown -R $USER:$USER /usr/local/hadoop

Step 5: Configure Environment Variables

Makes Hadoop commands available system-wide.

Commands:

Edit ~/.bashrc and add JAVA_HOME, HADOOP_HOME, PATH entries.
Run: source ~/.bashrc

Step 6: Configure JAVA_HOME

Hadoop needs the Java path.

Commands:

Edit hadoop-env.sh and set:
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

Step 7: Configure core-site.xml

Defines the default HDFS URI.

Commands:

Set fs.defaultFS = hdfs://localhost:9000

Step 8: Configure hdfs-site.xml

Defines NameNode/DataNode directories.

Commands:

Set dfs.replication=1 and configure namenode/datanode paths.

Step 9: Configure mapred-site.xml

Enables MapReduce on YARN.

Commands:

Set mapreduce.framework.name=yarn

Step 10: Configure yarn-site.xml

Enables shuffle service.

Commands:

Set yarn.nodemanager.aux-services=mapreduce_shuffle

Step 11: Configure workers

Defines worker nodes.

Commands:

localhost

Step 12: Create HDFS Directories

Creates storage locations.

Commands:

mkdir -p ~/hadoopdata/namenode
mkdir -p ~/hadoopdata/datanode

Step 13: Format NameNode

Initializes HDFS.

Commands:

hdfs namenode -format

Step 14: Start Hadoop

Starts HDFS and YARN services.

Commands:

start-dfs.sh
start-yarn.sh

Step 15: Verify

Confirms Hadoop services are running.

Commands:

jps

Step 16: Test HDFS

Commands:

```
hdfs dfs -mkdir /test
echo "Hello Hadoop" > sample.txt
hdfs dfs -put sample.txt /test
hdfs dfs -cat /test/sample.txt
```

Step 17: Web Interfaces

Commands:

HDFS: http://localhost:9870
YARN: http://localhost:8088

Step 1: Check Hadoop Daemons

Command

jps

Output

12345 NameNode

12456 DataNode

12567 SecondaryNameNode

12678 ResourceManager

12789 NodeManager

12890 Jps

Step 2: Create a Directory in HDFS

Command

hdfs dfs -mkdir /experiment1

Step 3: List the Root Directory

Command

hdfs dfs -ls /

Output

Found 2 items

drwxr-xr-x - jit supergroup 0 2026-08-03 11:00 /experiment1

drwxr-xr-x - jit supergroup 0 2026-08-03 10:30 /tmp

Step 4: Create Local Text Files

Command

echo "VTU Big Data Analytics Lab" > file1.txt

echo "Jyothy Institute of Technology" > file2.txt

Output

(No output)

Step 5: Verify Local Files

Command

Ls

Output

file1.txt

file2.txt

Step 6: Upload Files to HDFS

Command

hdfs dfs -put file1.txt /experiment1

hdfs dfs -put file2.txt /experiment1

Step 7: Verify Uploaded Files

Command

hdfs dfs -ls /experiment1

Output

Found 2 items

-rw-r--r-- 1 jit supergroup 27 2026-08-03 11:10 /experiment1/file1.txt

-rw-r--r-- 1 jit supergroup 32 2026-08-03 11:10 /experiment1/file2.txt

Step 8: Display File Content

Command

hdfs dfs -cat /experiment1/file1.txt

Output

VTU Big Data Analytics Lab

Step 9: Copy a File

Command

hdfs dfs -cp /experiment1/file1.txt /experiment1/file1_copy.txt

Step 10: Verify the Copied File

Command

hdfs dfs -ls /experiment1

Expected Output

Found 3 items

-rw-r--r-- 1 jit supergroup 27 2026-08-03 11:10 /experiment1/file1.txt

-rw-r--r-- 1 jit supergroup 27 2026-08-03 11:11 /experiment1/file1_copy.txt

-rw-r--r-- 1 jit supergroup 32 2026-08-03 11:10 /experiment1/file2.txt

Step 11: Rename a File

Command

hdfs dfs -mv /experiment1/file2.txt /experiment1/file2_renamed.txt

Step 12: Verify Renamed File

Command

hdfs dfs -ls /experiment1

Output

Found 3 items

-rw-r--r-- 1 jit supergroup 27 2026-08-03 11:10 /experiment1/file1.txt

-rw-r--r-- 1 jit supergroup 27 2026-08-03 11:11 /experiment1/file1_copy.txt

-rw-r--r-- 1 jit supergroup 32 2026-08-03 11:12 /experiment1/file2_renamed.txt

Step 13: Create a Local Directory

Command

mkdir retrieved

Step 14: Download a File from HDFS

Command

hdfs dfs -get /experiment1/file1.txt retrieved/

Step 15: Verify Downloaded File

Command

ls retrieved

Output

file1.txt

Step 16: View Downloaded File

Command

cat retrieved/file1.txt

Output

VTU Big Data Analytics Lab

Step 17: Delete Copied File

Command

hdfs dfs -rm /experiment1/file1_copy.txt

Output

Deleted /experiment1/file1_copy.txt

Step 18: Verify Deletion

Command

hdfs dfs -ls /experiment1

Output

Found 2 items

-rw-r--r-- 1 jit supergroup 27 2026-08-03 11:10 /experiment1/file1.txt

-rw-r--r-- 1 jit supergroup 32 2026-08-03 11:12 /experiment1/file2_renamed.txt

Step 19: Delete Remaining Files

Command

hdfs dfs -rm /experiment1/file1.txt

hdfs dfs -rm /experiment1/file2_renamed.txt

Output

Deleted /experiment1/file1.txt

Deleted /experiment1/file2_renamed.txt

Step 20: Verify Directory is Empty

Command

hdfs dfs -ls /experiment1

Output

Found 0 items

Step 21: Remove Empty Directory

Command

hdfs dfs -rmdir /experiment1

Step 22: Verify Directory Removal

Command

hdfs dfs -ls /

Output

Found 1 items

drwxr-xr-x   - jit supergroup   0 2026-08-03 10:30 /tmp

Program 2

Develop a MapReduce Program to Implement Matrix Multiplication

Code

Input File (input.txt)

A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8

This represents:

Matrix A

1 2
3 4

Matrix B

5 6
7 8

Mapper Program (mapper.py)

```
#!/usr/bin/env python3

import sys

N = 2

for line in sys.stdin:
    matrix, i, j, value = line.strip().split(",")

    i = int(i)
```

```
    j = int(j)
    value = float(value)

    if matrix == "A":
        for col in range(N):
            print(f"{i},{col}\tA,{j},{value}")
    else:
        for row in range(N):
            print(f"{row},{j}\tB,{i},{value}")
```

Reducer Program (reducer.py)

```
#!/usr/bin/env python3

import sys

current_key = None
A = {}
B = {}

def multiply(A, B):
    total = 0
    for k in A:
        if k in B:
            total += A[k] * B[k]
    return total

for line in sys.stdin:
    key, value = line.strip().split("\t")

    if current_key != key:
        if current_key is not None:
            print(f"{current_key}\t{multiply(A,B)}")

        current_key = key
        A = {}
        B = {}

    matrix, index, val = value.split(",")

    if matrix == "A":
```

```
        A[int(index)] = float(val)
    else:
        B[int(index)] = float(val)

if current_key is not None:
    print(f"{current_key}\t{multiply(A,B)}")
```

Execution Commands

Step 1: Start Hadoop Services

Start HDFS and YARN, then verify that all Hadoop daemons are running.

```
start-dfs.sh
start-yarn.sh
jps
```

Expected processes:

- NameNode
- DataNode
- SecondaryNameNode
- ResourceManager
- NodeManager

Step 2: Create the Project Directory

Create a project folder and move into it.

```
mkdir MatrixMultiplication
cd MatrixMultiplication
```

Step 3: Create the Required Files

Create the input file, mapper program, and reducer program.

```
nano input.txt
nano mapper.py
nano reducer.py
```

Step 4: Make Python Files Executable

Grant execute permission to the mapper and reducer.

```
chmod +x mapper.py
chmod +x reducer.py
```

Step 5: Verify the Files

Check that all required files are present.

```
ls
```

Expected output:

```
input.txt
mapper.py
reducer.py
```

Step 6: Test the Program Locally

Execute the MapReduce program without Hadoop.

```
cat input.txt | python3 mapper.py | sort | python3 reducer.py
```

Expected output:

```
0,0    19.0
0,1    22.0
1,0    43.0
1,1    50.0
```

Step 7: Create the HDFS Directory

Create a directory in HDFS to store the input file.

```
hdfs dfs -mkdir -p /matrix
```

Step 8: Upload the Input File

Copy the input file to HDFS.

hdfs dfs -put input.txt /matrix

Step 9: Verify the Uploaded File

Confirm that the file is available in HDFS.

hdfs dfs -ls /matrix

Expected output:

Found 1 items
-rw-r--r--   1 user supergroup   64 /matrix/input.txt

Step 10: Remove the Existing Output Directory

Delete the previous output directory before rerunning the program.

hdfs dfs -rm -r /matrix/output

If the directory does not exist, Hadoop displays:

rm: `/matrix/output': No such file or directory

This can be ignored.

Step 11: Execute the Hadoop Streaming Job

Run the Matrix Multiplication MapReduce program.

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-files mapper.py,reducer.py \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-input /matrix/input.txt \
-output /matrix/output
```

Successful execution ends with:

Job completed successfully
Output directory: /matrix/output

Step 12: Display the Output

View the generated result stored in HDFS.

hdfs dfs -cat /matrix/output/part-00000

Expected output:

0,0 19.0
0,1 22.0
1,0 43.0
1,1 50.0

Result Matrix

19 22
43 50

Program 3

Develop a MapReduce Program to Mine Weather Data and Display Appropriate Messages Indicating the Weather Conditions of the Day

Code

Input File (input.txt)

```
2026-07-22,35
2026-07-23,28
2026-07-24,18
2026-07-25,10
2026-07-26,32
```

Mapper Program (mapper.py)

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    line = line.strip()
    if line:
        print(line)
```

Reducer Program (reducer.py)

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    date, temp = line.strip().split(",")
    temp = float(temp)

    if temp >= 30:
        condition = "Hot"
    elif temp >= 20:
```

```
        condition = "Warm"
    elif temp >= 15:
        condition = "Cool"
    else:
        condition = "Cold"

    print(f"{date}\t{temp}°C\t{condition}")
```

Execution Commands

Step 1: Start Hadoop services.

start-dfs.sh
start-yarn.sh
jps

Step 2: Create the project directory.

mkdir WeatherData
cd WeatherData

Step 3: Create the required files.

nano input.txt
nano mapper.py
nano reducer.py

Step 4: Make the Python files executable.

chmod +x mapper.py
chmod +x reducer.py

Step 5: Verify the files.

ls

Expected output:

input.txt
mapper.py
reducer.py

Step 6: Test the program locally.

cat input.txt | python3 mapper.py | sort | python3 reducer.py

Step 7: Create an HDFS directory.

hdfs dfs -mkdir -p /weather

Step 8: Upload the input file to HDFS.

hdfs dfs -put input.txt /weather

Step 9: Verify the uploaded file.

hdfs dfs -ls /weather

Step 10: Remove the existing output directory (if any).

hdfs dfs -rm -r /weather/output

Step 11: Run the Hadoop Streaming job.

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-files mapper.py,reducer.py \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-input /weather/input.txt \
-output /weather/output
```

Step 12: Display the ou-tput.

hdfs dfs -cat /weather/output/part-00000

Output

Local Execution Output

```
2026-07-22    35.0°C    Hot
2026-07-23    28.0°C    Warm
2026-07-24    18.0°C    Cool
2026-07-25    10.0°C    Cold
2026-07-26    32.0°C    Hot
```

Program 4

Develop a MapReduce Program to Find the Tags Associated with Each Movie by Analyzing MovieLens Data

Code

Input File (input.txt)

```
1,Toy Story,Animation
1,Toy Story,Comedy
2,Avatar,Sci-Fi
2,Avatar,Action
3,Titanic,Romance
3,Titanic,Drama
1,Toy Story,Family
2,Avatar,Adventure
```

Mapper Program (mapper.py)

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    movie_id, movie_name, tag = line.strip().split(",")
    print(f"{movie_name}\t{tag}")
```

Reducer Program (reducer.py)

```
#!/usr/bin/env python3

import sys

current_movie = None
tags = []

for line in sys.stdin:
    movie, tag = line.strip().split("\t")
```

```
    if current_movie != movie:
        if current_movie is not None:
            print(f"{current_movie}\t{', '.join(tags)}")

        current_movie = movie
        tags = []

    tags.append(tag)

if current_movie is not None:
    print(f"{current_movie}\t{', '.join(tags)}")
```

Execution Commands

Step 1: Start Hadoop services.

```
start-dfs.sh
start-yarn.sh
jps
```

Step 2: Create the project directory.

```
mkdir MovieTags
cd MovieTags
```

Step 3: Create the required files.

```
nano input.txt
nano mapper.py
nano reducer.py
```

Step 4: Make the Python files executable.

```
chmod +x mapper.py
chmod +x reducer.py
```

Step 5: Verify the files.

ls

Expected output:

input.txt
mapper.py
reducer.py

Step 6: Test the program locally.

cat input.txt | python3 mapper.py | sort | python3 reducer.py

Step 7: Create an HDFS directory.

hdfs dfs -mkdir -p /movietags

Step 8: Upload the input file to HDFS.

hdfs dfs -put input.txt /movietags

Step 9: Verify the uploaded file.

hdfs dfs -ls /movietags

Step 10: Remove the existing output directory (if any).

hdfs dfs -rm -r /movietags/output

Step 11: Run the Hadoop Streaming job.

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar \
-files mapper.py,reducer.py \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-input /movietags/input.txt \
-output /movietags/output
```

Step 12: Display the output.

hdfs dfs -cat /movietags/output/part-00000

Output

Local Execution Output

Avatar Action, Adventure, Sci-Fi
Titanic Drama, Romance
Toy Story Animation, Comedy, Family

Program 5

Implement Functions: Count Sort Limit— Skip Aggregate using MongoDB

Code

```
from pymongo import MongoClient

# Connect to MongoDB
client = MongoClient("mongodb://localhost:27017/")

# Create Database
db = client["college"]

# Create Collection
students = db["students"]

# Delete existing records
students.delete_many({})

# Insert Sample Data
students.insert_many([
    {"name": "Alice", "age": 20, "marks": 85},
    {"name": "Bob", "age": 22, "marks": 75},
    {"name": "Charlie", "age": 21, "marks": 90},
    {"name": "David", "age": 23, "marks": 60},
    {"name": "Eva", "age": 20, "marks": 95},
    {"name": "Frank", "age": 22, "marks": 70}
])

# COUNT
print("\nCOUNT")
count = students.count_documents({})
print("Total Students:", count)

# SORT
print("\nSORT (Marks Descending)")
for student in students.find().sort("marks", -1):
```

```
    print(student)

# LIMIT
print("\nLIMIT (Top 3 Students)")
for student in students.find().sort("marks", -1).limit(3):
    print(student)

# SKIP
print("\nSKIP (Skip First 2 Students)")
for student in students.find().sort("marks", -1).skip(2):
    print(student)

# AGGREGATE
print("\nAGGREGATE")

pipeline = [
    {
        "$group": {
            "_id": None,
            "AverageMarks": {"$avg": "$marks"},
            "HighestMarks": {"$max": "$marks"},
            "LowestMarks": {"$min": "$marks"},
            "TotalStudents": {"$sum": 1}
        }
    }
]

result = students.aggregate(pipeline)

for r in result:
    print(r)

client.close()
```

Steps to Execute

Step 1: Update Ubuntu Packages

```
sudo apt update
```

Step 2: Install MongoDB

sudo apt install mongodb -y

Step 3: Start MongoDB Service

sudo systemctl start mongodb

Step 4: Enable MongoDB

sudo systemctl enable mongodb

Step 5: Check MongoDB Status

sudo systemctl status mongodb

Step 6: Install Python pip

sudo apt install python3-pip -y

Step 7: Install PyMongo

pip3 install pymongo

Step 8: Verify PyMongo Installation

python3 -c "import pymongo; print(pymongo.__version__)"

Step 9: Create Python Program

nano mongodb_lab.py

Step 10: Run the Program

python3 mongodb_lab.py

Output

COUNT
Total Students: 6

SORT (Marks Descending)
{'name': 'Eva', 'age': 20, 'marks': 95}
{'name': 'Charlie', 'age': 21, 'marks': 90}
{'name': 'Alice', 'age': 20, 'marks': 85}
{'name': 'Bob', 'age': 22, 'marks': 75}
{'name': 'Frank', 'age': 22, 'marks': 70}
{'name': 'David', 'age': 23, 'marks': 60}

LIMIT (Top 3 Students)
{'name': 'Eva', 'age': 20, 'marks': 95}
{'name': 'Charlie', 'age': 21, 'marks': 90}
{'name': 'Alice', 'age': 20, 'marks': 85}

SKIP (Skip First 2 Students)
{'name': 'Alice', 'age': 20, 'marks': 85}
{'name': 'Bob', 'age': 22, 'marks': 75}
{'name': 'Frank', 'age': 22, 'marks': 70}
{'name': 'David', 'age': 23, 'marks': 60}

AGGREGATE
{
 '_id': None,
 'AverageMarks': 79.16666666666667,
 'HighestMarks': 95,
 'LowestMarks': 60,
 'TotalStudents': 6
}

MongoDB Shell Commands (Verification)

Open MongoDB shell:

mongosh

Show databases:

show dbs

Use the database:

use college

Show collections:

show collections

Display all documents:

db.students.find()

Count documents:

db.students.countDocuments()

Sort by marks (descending):

db.students.find().sort({marks:-1})

Display top 3 documents:

db.students.find().sort({marks:-1}).limit(3)

Skip first 2 documents:

db.students.find().sort({marks:-1}).skip(2)

Aggregate statistics:

```
db.students.aggregate([
{
  $group:{
    _id:null,
    AverageMarks:{$avg:"$marks"},
    HighestMarks:{$max:"$marks"},
    LowestMarks:{$min:"$marks"},
    TotalStudents:{$sum:1}
  }
}
])
```

Experiment 6

Write Pig Latin scripts to sort, group, join, project, and filter the data.

Input File 1: students.csv

1,Alice,CSE,85
2,Bob,ECE,75
3,Charlie,CSE,90
4,David,EEE,60
5,Eva,CSE,95
6,Frank,ECE,70

Input File 2: department.csv

CSE,Computer Science
ECE,Electronics
EEE,Electrical

Pig Script (student_operations.pig)

```
students = LOAD 'students.csv'
USING PigStorage(',')
AS (
    id:int,
    name:chararray,
    dept:chararray,
    marks:int
);

-- Project
projected = FOREACH students GENERATE id, name, marks;
DUMP projected;

-- Filter
filtered = FILTER students BY marks > 80;
```

```
DUMP filtered;

-- Sort
sorted = ORDER students BY marks DESC;
DUMP sorted;

-- Group
grp = GROUP students BY dept;
result = FOREACH grp GENERATE group, COUNT(students);
DUMP result;

-- Load Department File
dept = LOAD 'department.csv'
USING PigStorage(',')
AS (
    dept:chararray,
    dept_name:chararray
);

-- Join
joined = JOIN students BY dept, dept BY dept;
DUMP joined;
```

Steps to Execute

Step 1: Start Hadoop Services

```
start-dfs.sh
start-yarn.sh
```

Step 2: Create Input Directory in HDFS

```
hdfs dfs -mkdir -p /user/$USER/input
```

Step 3: Copy Input Files to HDFS

```
hdfs dfs -put students.csv /user/$USER/input/
hdfs dfs -put department.csv /user/$USER/input/
```

Step 4: Create Pig Script

nano student_operations.pig

Paste the Pig script into the file.

Save the file:

- **Ctrl + O**
- **Enter**
- **Ctrl + X**

Step 5: Start Apache Pig

pig

or execute the script directly:

pig student_operations.pig

> **Note:** If using HDFS, update the `LOAD` statements as follows:

```
students = LOAD '/user/$USER/input/students.csv'
USING PigStorage(',')
AS (
    id:int,
    name:chararray,
    dept:chararray,
    marks:int
);

dept = LOAD '/user/$USER/input/department.csv'
USING PigStorage(',')
AS (
    dept:chararray,
    dept_name:chararray
        );
```

Output

Project

(1,Alice,85)
(2,Bob,75)
(3,Charlie,90)
(4,David,60)
(5,Eva,95)
(6,Frank,70)

Filter (Marks > 80)

(1,Alice,CSE,85)
(3,Charlie,CSE,90)
(5,Eva,CSE,95)

Sort (Descending Marks)

(5,Eva,CSE,95)
(3,Charlie,CSE,90)
(1,Alice,CSE,85)
(2,Bob,ECE,75)
(6,Frank,ECE,70)
(4,David,EEE,60)

Group and Count

(CSE,3)
(ECE,2)
(EEE,1)

Join

(1,Alice,CSE,85,CSE,Computer Science)
(2,Bob,ECE,75,ECE,Electronics)
(3,Charlie,CSE,90,CSE,Computer Science)
(4,David,EEE,60,EEE,Electrical)
(5,Eva,CSE,95,CSE,Computer Science)
(6,Frank,ECE,70,ECE,Electronics)

Experiment 7

Use Hive to create, alter, and drop databases, tables, views, functions, and indexes.

HiveQL Script

```
CREATE DATABASE college;

USE college;

CREATE TABLE students(
   id INT,
   name STRING,
   dept STRING,
   marks INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

ALTER TABLE students
ADD COLUMNS(age INT);

ALTER TABLE students
RENAME TO student_details;

CREATE VIEW cse_students AS
SELECT *
FROM student_details
WHERE dept='CSE';

SHOW FUNCTIONS;

SELECT UPPER('hadoop');

CREATE INDEX student_index
```

```
ON TABLE student_details(name)
AS 'COMPACT'
WITH DEFERRED REBUILD;

ALTER INDEX student_index
ON student_details
REBUILD;

SHOW INDEXES ON student_details;

DROP VIEW cse_students;

DROP TABLE student_details;

DROP DATABASE college CASCADE;
```

Steps to Execute

Step 1: Start Hadoop

start-dfs.sh
start-yarn.sh

Step 2: Verify Hadoop Services

jps

Expected Output

NameNode
DataNode
SecondaryNameNode
ResourceManager
NodeManager
Jps

Step 3: Start Hive

hive

You will see:

hive>

Step 4: Create Hive Script

nano hive_lab.hql

Paste the HiveQL commands into the file.

Save the file:

- **Ctrl + O**
- **Enter**
- **Ctrl + X**

Step 5: Execute the Hive Script

hive -f hive_lab.hql

Output

Create Database

CREATE DATABASE college;
OK
Time taken: 0.x seconds

Show Databases

SHOW DATABASES;
default
college

Use Database

USE college;
OK

Create Table

SHOW TABLES;
students

Describe Table

DESCRIBE students;
id int
name string
dept string
marks int

Alter Table (Add Column)

DESCRIBE students;
id int
name string
dept string
marks int
age int

Rename Table

SHOW TABLES;
student_details

Create View

SELECT * FROM cse_students;
(Output displays all records where department = 'CSE')

Hive Function

SELECT UPPER('hadoop');
HADOOP

Create and Show Index

SHOW INDEXES ON student_details;
student_index

Drop View

DROP VIEW cse_students;
OK

Drop Table

DROP TABLE student_details;
OK

Drop Database

DROP DATABASE college CASCADE;
OK

Implement a Word Count Program Using Hadoop MapR

Input File (input.txt)

Hadoop is a big data framework
Hadoop is used for distributed computing
Big data is processed using Hadoop

Mapper Program (mapper.py)

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    line = line.strip().lower()
    words = line.split()

    for word in words:
        print(f"{word}\t1")
```

Reducer Program (reducer.py)

```
#!/usr/bin/env python3

import sys

current_word = None
current_count = 0

for line in sys.stdin:
    line = line.strip()

    word, count = line.split('\t')
```

```
    count = int(count)

    if current_word == word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

if current_word:
    print(f"{current_word}\t{current_count}")
```

Execution Commands

Step 1: Start Hadoop Services

start-dfs.sh
start-yarn.sh
jps

Step 2: Create the Project Directory

mkdir WordCount
cd WordCount

Step 3: Create the Input File

nano input.txt

Paste the input data and save.

Step 4: Create the Mapper Program

nano mapper.py

Step 5: Create the Reducer Program

nano reducer.py

Step 6: Give Execute Permission

chmod +x mapper.py
chmod +x reducer.py

Step 7: Test the Mapper Locally

cat input.txt | python3 mapper.py

Step 8: Test the Reducer Locally

cat input.txt | python3 mapper.py | sort | python3 reducer.py

Step 9: Create an HDFS Directory

hdfs dfs -mkdir /wordcount

Verify:

hdfs dfs -ls /

Step 10: Upload the Input File to HDFS

hdfs dfs -put input.txt /wordcount/

Verify:

hdfs dfs -ls /wordcount

Step 11: Remove Previous Output (if any)

hdfs dfs -rm -r /wordcount_output

Step 12: Run the Hadoop Streaming Job

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-input /wordcount/input.txt \
-output /wordcount_output \
-mapper mapper.py \
-reducer reducer.py \
-file mapper.py \
-file reducer.py

Step 13: View the Output

hdfs dfs -cat /wordcount_output/part-00000

Expected Output

a 1
big 2
computing 1
data 2
distributed 1
for 1
framework 1
hadoop 3
is 3
processed 1
used 1
using 1

Use CDH (Cloudera Distribution for Hadoop) and HUE (Hadoop User Interface) to Analyze Data and Generate Reports for Sample Datasets

Input File (sales.txt)

101,Laptop,5,50000

102,Mouse,10,500

103,Keyboard,7,1200

104,Laptop,3,50000

105,Mouse,8,500

106,Keyboard,5,1200

107,Laptop,2,50000

Format:

OrderID,Product,Quantity,Price

Mapper Program (mapper.py)

```
#!/usr/bin/env python3


import sys


for line in sys.stdin:
    line = line.strip()


    if line:
        orderid, product, qty, price = line.split(",")


        amount = int(qty) * int(price)


        print(f"{product}\t{amount}")
```

Reducer Program (reducer.py)

```
#!/usr/bin/env python3
```

```
import sys

current_product = None
current_total = 0

for line in sys.stdin:
    product, amount = line.strip().split("\t")
    amount = int(amount)

    if current_product == product:
        current_total += amount
    else:
        if current_product:
            print(f"{current_product}\t{current_total}")

        current_product = product
        current_total = amount

if current_product:
    print(f"{current_product}\t{current_total}")
```

Execution Commands

Step 1: Start Hadoop Services

start-dfs.sh

start-yarn.sh

jps

Step 2: Create Project Directory

mkdir SalesReport

cd SalesReport

Step 3: Create Files

nano sales.txt

nano mapper.py

nano reducer.py

Step 4: Give Execute Permission

chmod +x mapper.py

chmod +x reducer.py

Step 5: Test Locally

cat sales.txt | python3 mapper.py

cat sales.txt | python3 mapper.py | sort | python3 reducer.py

Step 6: Create HDFS Directory

hdfs dfs -mkdir /sales

Step 7: Upload Dataset

hdfs dfs -put sales.txt /sales/

Verify

hdfs dfs -ls /sales

Step 8: Remove Previous Output

hdfs dfs -rm -r /sales_output

Step 9: Execute Hadoop Streaming

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \

-input /sales/sales.txt \

-output /sales_output \

-mapper mapper.py \

-reducer reducer.py \

-file mapper.py \

-file reducer.py

Step 10: View Output

hdfs dfs -cat /sales_output/part-00000

Output

Keyboard 14400

Laptop 500000

Mouse 9000